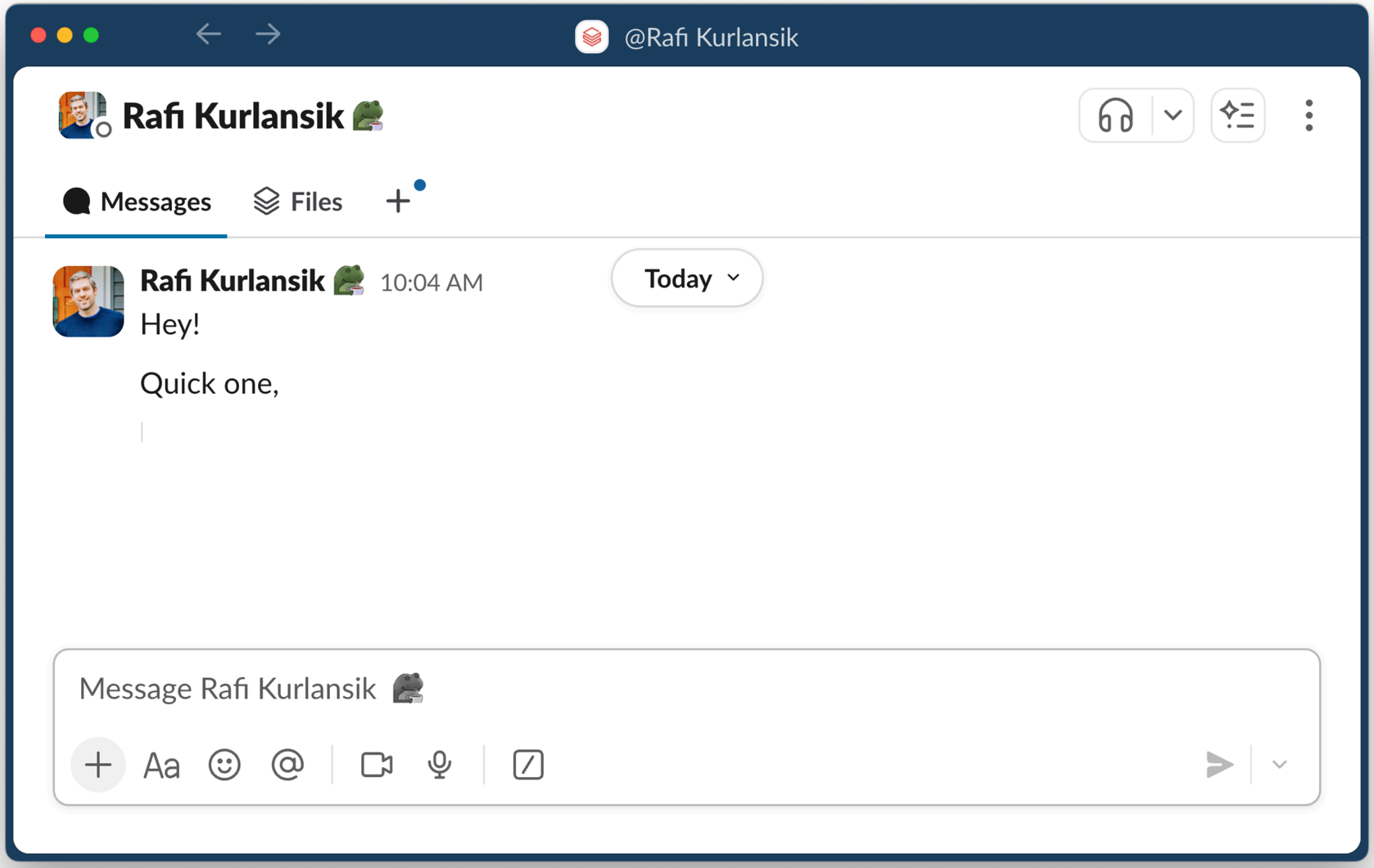
databricks

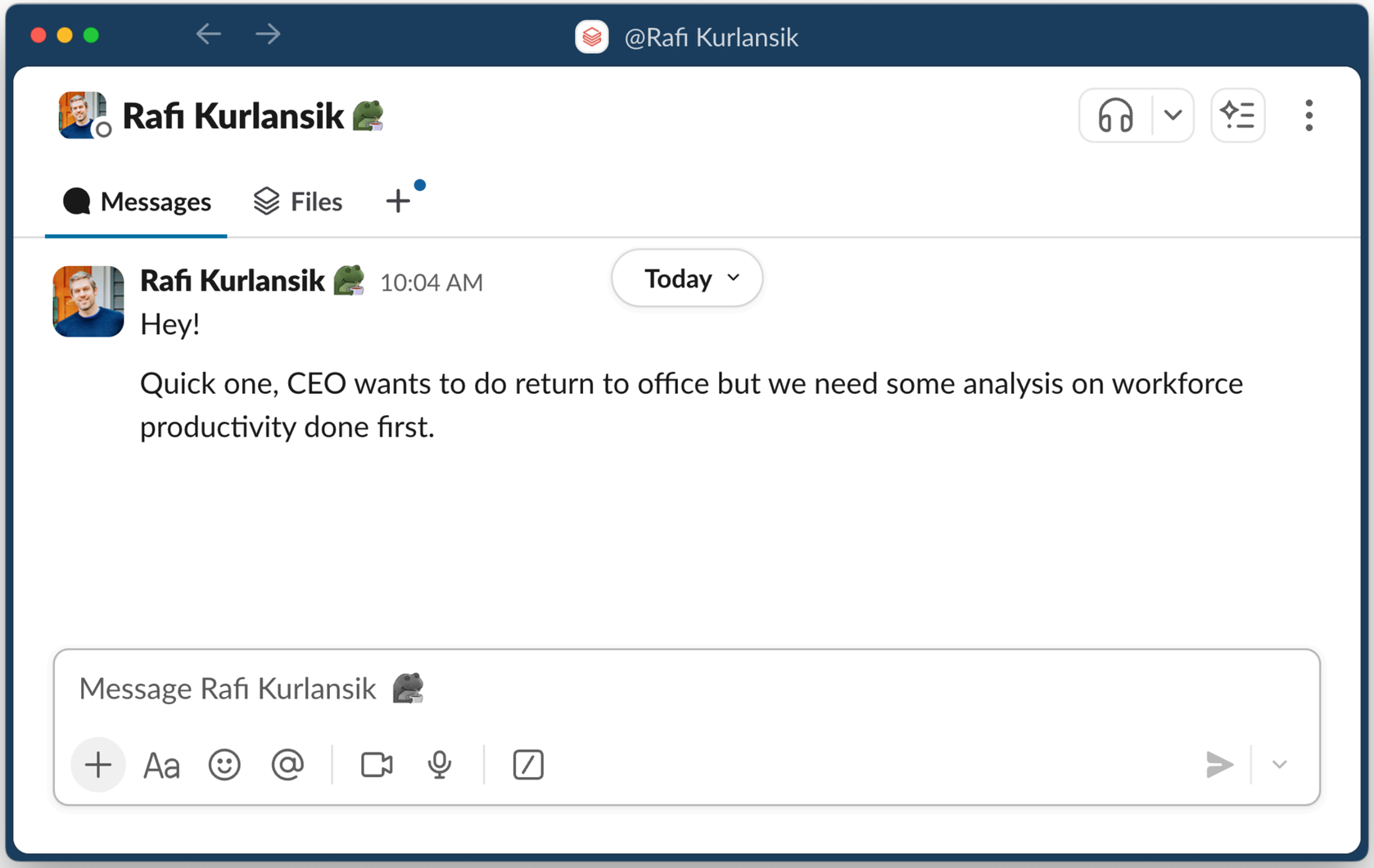
From SDKs to Agents:
Building with R & 🐍
on Databricks

Zac Davies, Rafi Kurlansik
Posit::conf(25)

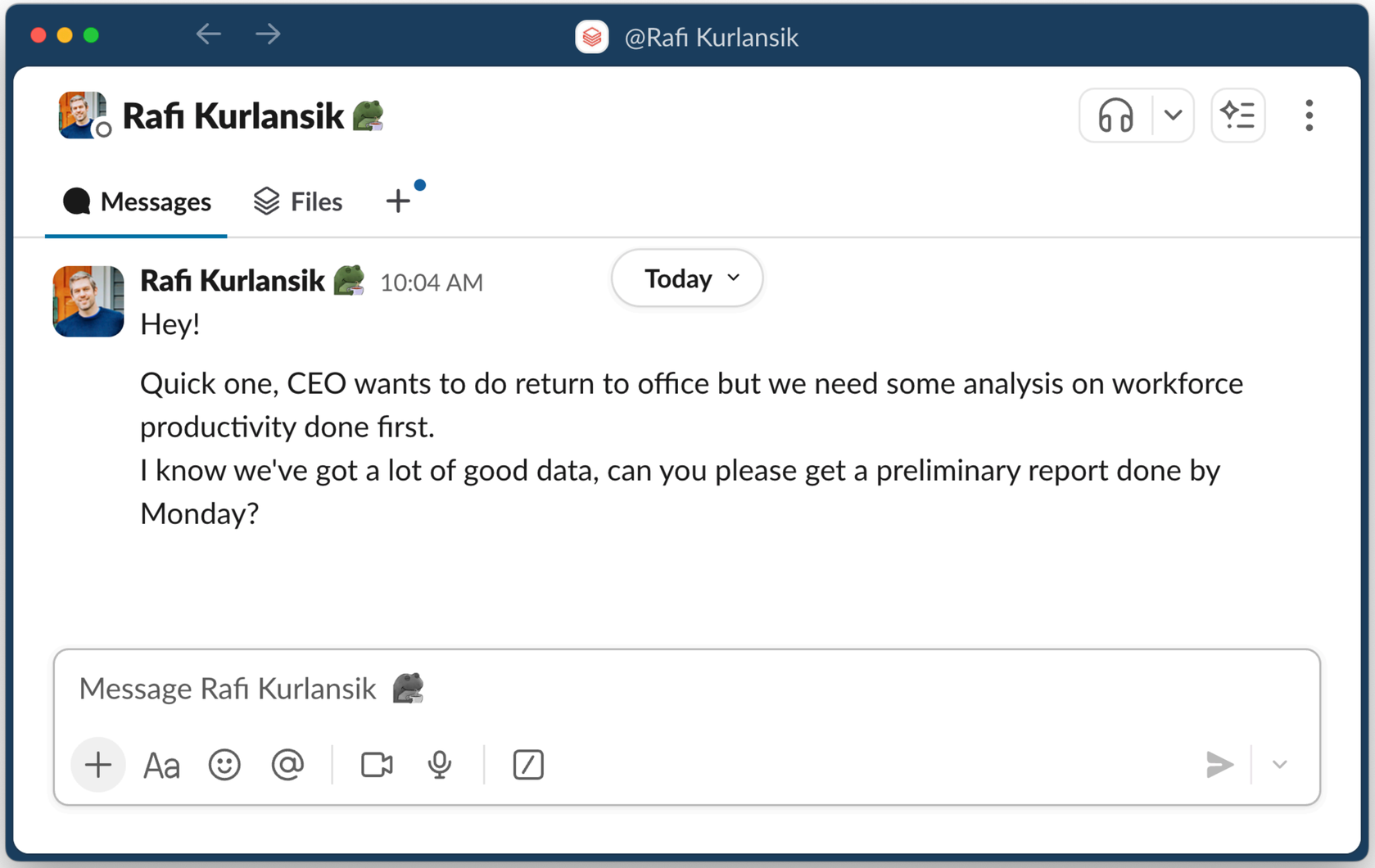
A typical day …



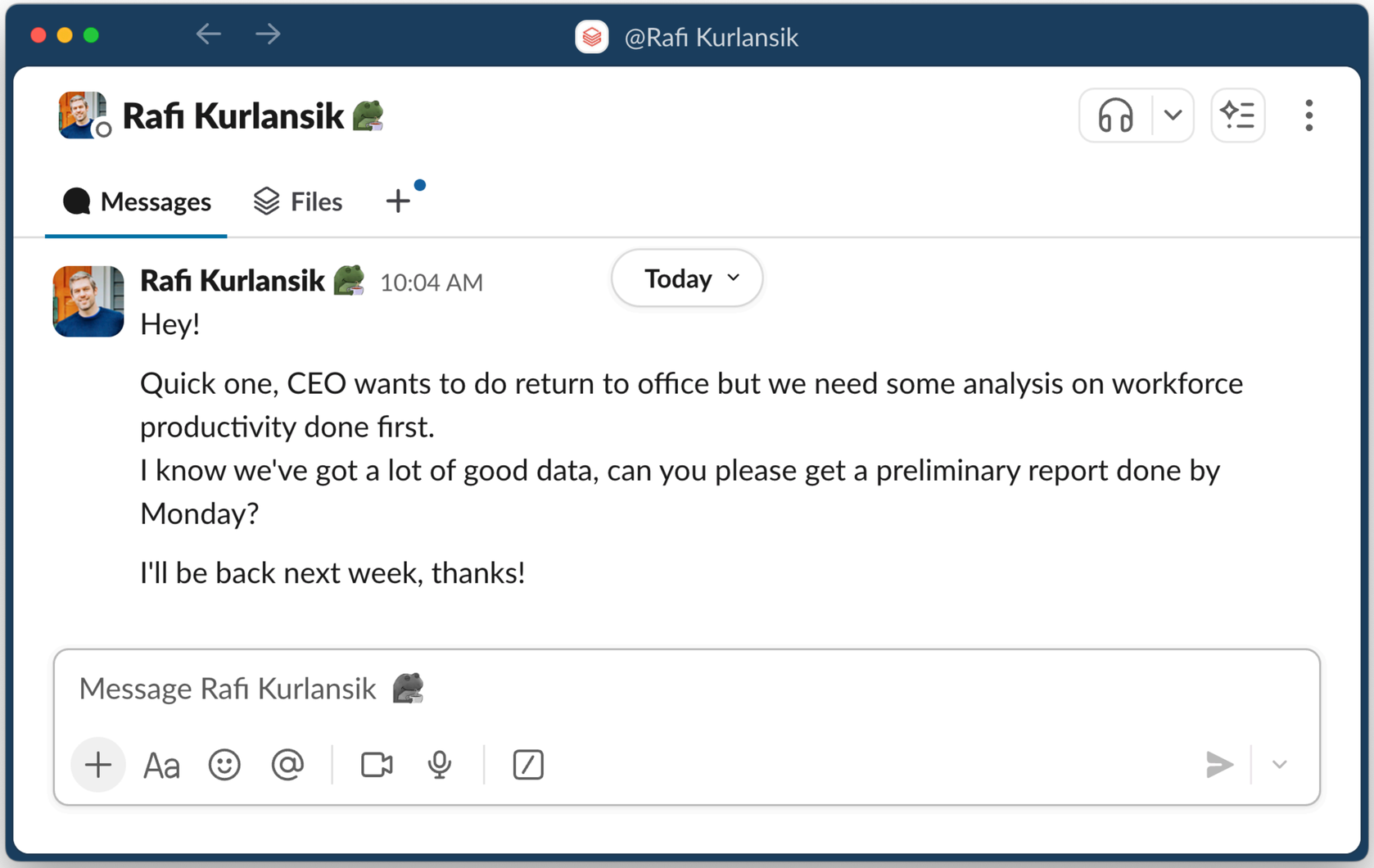
A typical day …



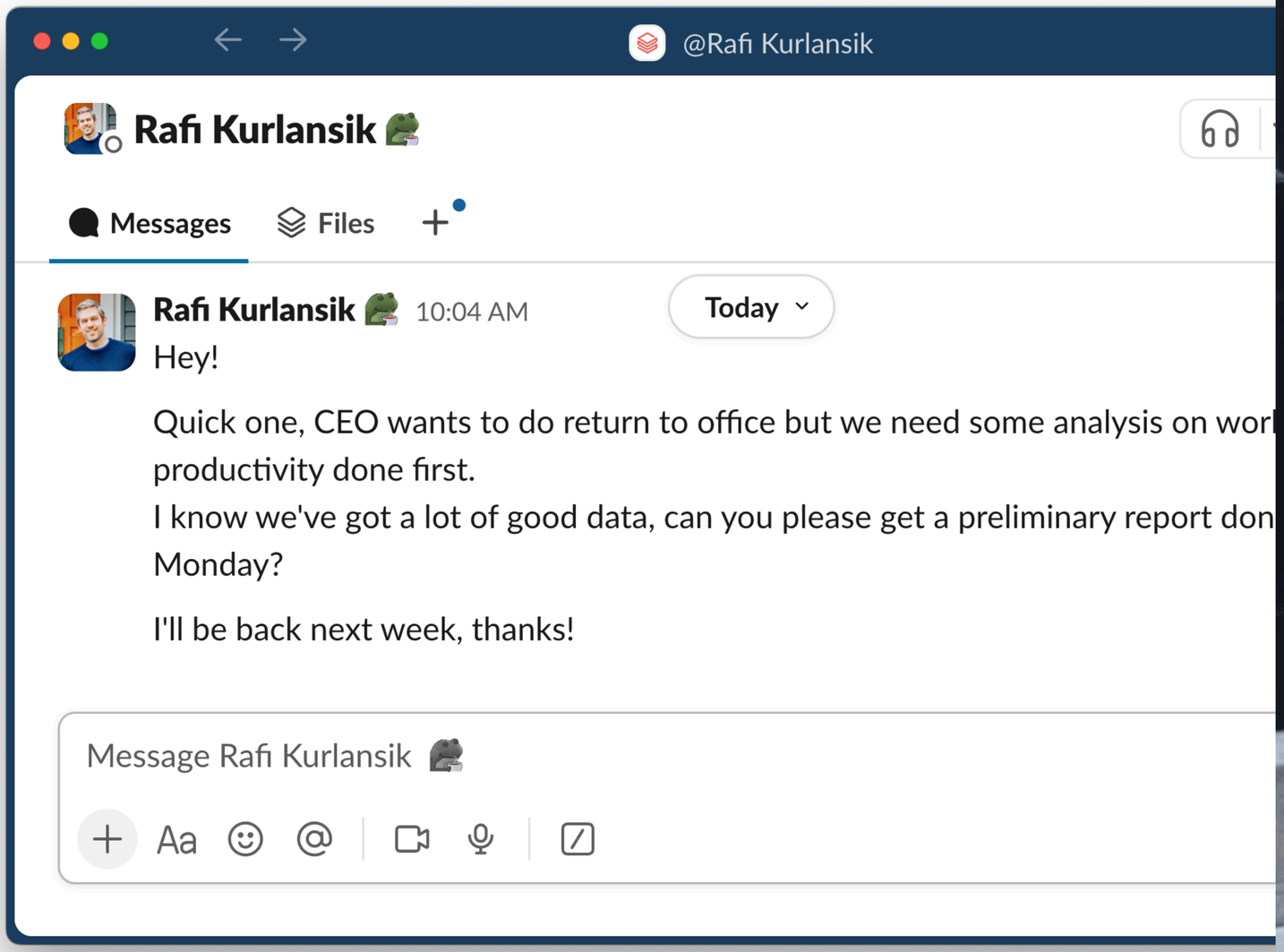
A typical day …



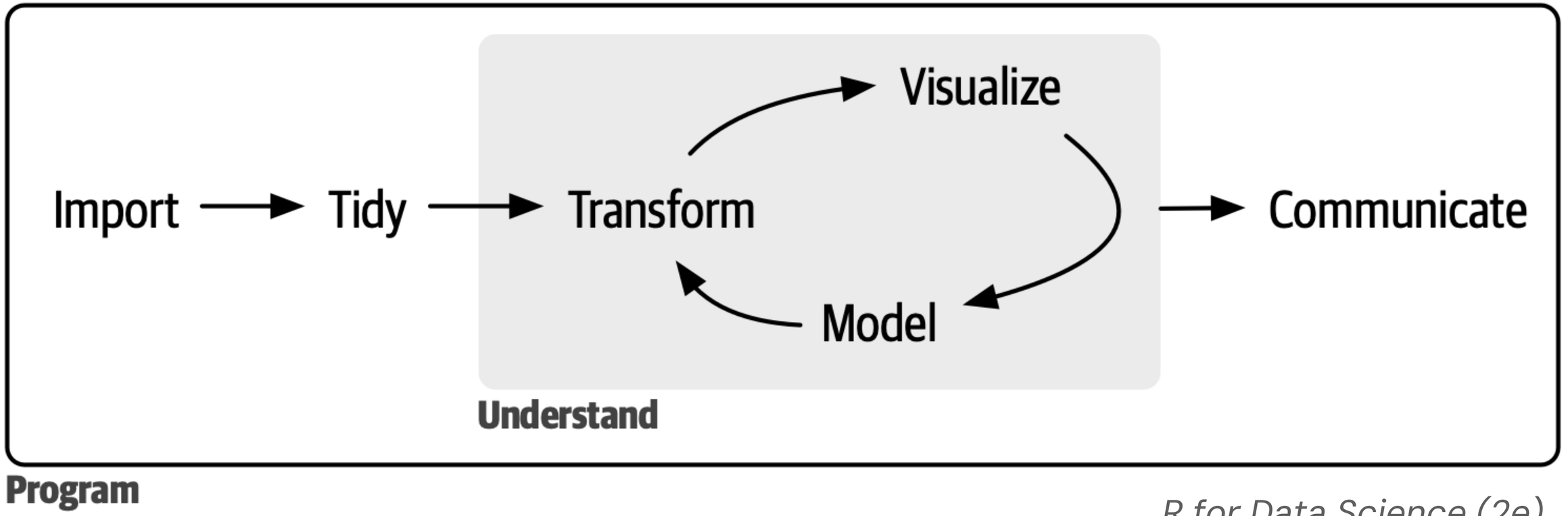
A typical day …



A typical day …



A process we're familiar with



R for Data Science (2e)

Preview | Render on Save | Source | Visual

discovery-analysis.qmd

Normal

```
---
title: "Discovery Analysis"
format: html
---
```

```
{r}
1 library(brickster)
2 library(tidyverse)
3 library(dbplyr)
4 library(DBI)
```

Data Discovery

First, let's connect to Databricks and exp

CONSOLE | TERMINAL | OUTPUT | PROBLEM

~/Documents/projects/conf25-agents

```
- The project is out-of-sync -- use `rer
>
```

RTO Productivity Analysis: Ke

databricks.atlassian.net/wiki/s...

Work | Relaunch to update

databricks | Search | + Create

Zachary Davies / RTO Productivity Analysis: Key Findings

Share

RTO Productivity Analysis: Key Findings

Owned by Zac Davies (FE Direct - International & Unassigned)
Last updated: just a moment ago • 2 min read • Analytics

RTO Productivity Analysis: Key Findings

Executive Summary

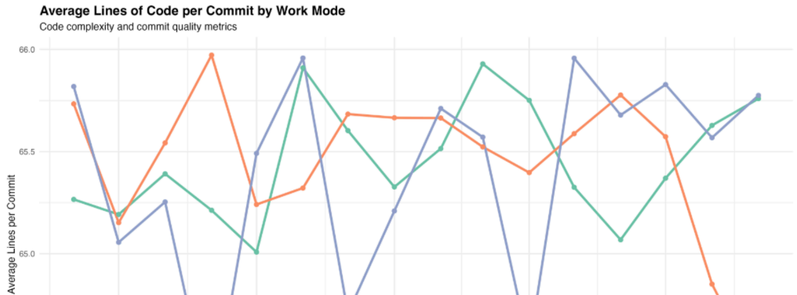
Our analysis of workforce productivity across different work modes reveals **no meaningful performance differences** between remote, hybrid, and colocated employees. Across all key metrics - development velocity, issue resolution efficiency, and customer service quality - work location appears to have minimal impact on productivity outcomes.

Dataset 1: GitHub Development Productivity

Key Findings

GitHub development metrics show virtually identical performance across all work modes:

Work Mode	Daily Commits	Code Quality Score	PR Review Time (hours)
Remote	2.1	4.2/5.0	18.2
Hybrid	2.0	4.1/5.0	18.9
Colocated	2.0	4.1/5.0	18.6



How realistic was that?

Data Security

- 3rd party providers (e.g. Anthropic)
- Assistant must respect data governance policies

Resource Constraints

- Tokens are expensive
- Do you have enough compute?

Complexity

- Multiple systems involved
- Authentication

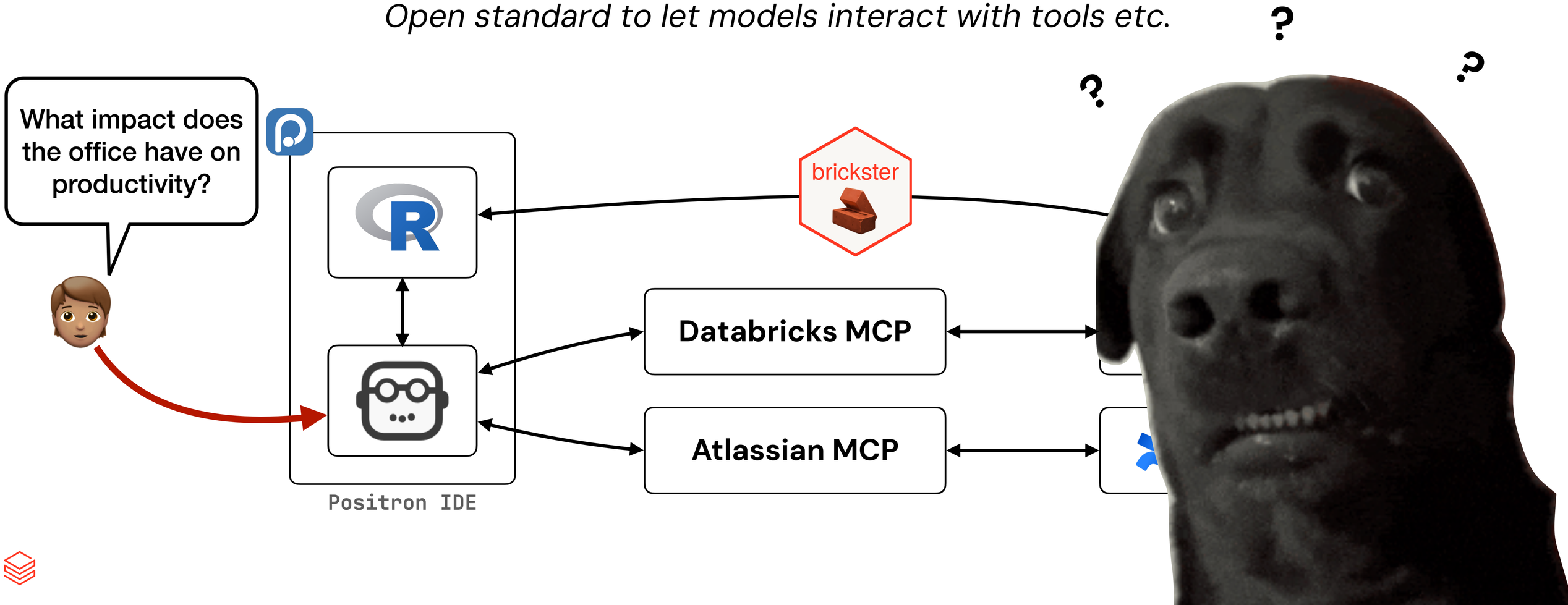
To understand if it’s realistic, let’s talk about what happened

What actually happened?

High level view

✨ **MCP:** Model Context Protocol ✨

Open standard to let models interact with tools etc.



Creating a tool

Allow LLM to discover resources in Databricks

```
# list all accessible SQL warehouses
warehouses ← brickster::db_sql_warehouse_list()

# extract required fields
purrr::map(warehouses, ~ .x[c("id", "name", "size", "creator_name", “state")])
```

```
[[1]]
[[1]]$id
[1] "c6a206525918f6eb"

[[1]]$name
[1] "Serverless Starter Warehouse"

[[1]]$size
[1] "XXSMALL"

[[1]]$creator_name
[1] "me@zacdav.com"

[[1]]$state
[1] "STOPPED"
```

Creating a tool

Annotate function with metadata


```r
db_warehouse_discovery ← ellmer::tool(
  fun = function() {
    warehouses ← brickster::db_sql_warehouse_list()
    purrr::map(
      warehouses,
      ~ .x[c("id", "name", "size", "creator_name", "state")]
    )
  },
  name = "list_databricks_warehouses",
  description = "Returns metadata for available warehouses: IDs, names, state, size, etc.",
  arguments = list(),
  annotations = ellmer::tool_annotations(
    title = "Databricks Warehouse Discovery Tool",
    read_only_hint = TRUE,
    open_world_hint = FALSE
  )
)
```

Creating a tool

Annotate function with metadata

```
db_warehouse_discovery ← ellmer::tool(
  fun = function() {
    warehouses ← brickster::db_sql_warehouse_list()
    purrr::map(
      warehouses,
      ~ .x[c("id", "name", "size", "creator_name", "state")]
    )
  },
  name = "list_databricks_warehouses",
  description = "Returns metadata for available warehouses: IDs, names, state, size, etc.",
  arguments = list(),
  annotations = ellmer::tool_annotations(
    title = "Databricks Warehouse Discovery Tool",
    read_only_hint = TRUE,
    open_world_hint = FALSE
  )
)
```

Code from before

Creating a tool

Annotate function with metadata

```r
db_warehouse_discovery ← ellmer::tool(
  fun = function() {
    warehouses ← brickster::db_sql_warehouse_list()
    purrr::map(
      warehouses,
      ~ .x[c("id", "name", "size", "creator_name", "state")]
    )
  },
  name = "list_databricks_warehouses",
  description = "Returns metadata for available warehouses: IDs, names, state, size, etc.",
  arguments = list(),
  annotations = ellmer::tool_annotations(
    title = "Databricks Warehouse Discovery Tool",
    read_only_hint = TRUE,
    open_world_hint = FALSE
  )
)
```

Metadata

“How to use the tool”

MCP time!

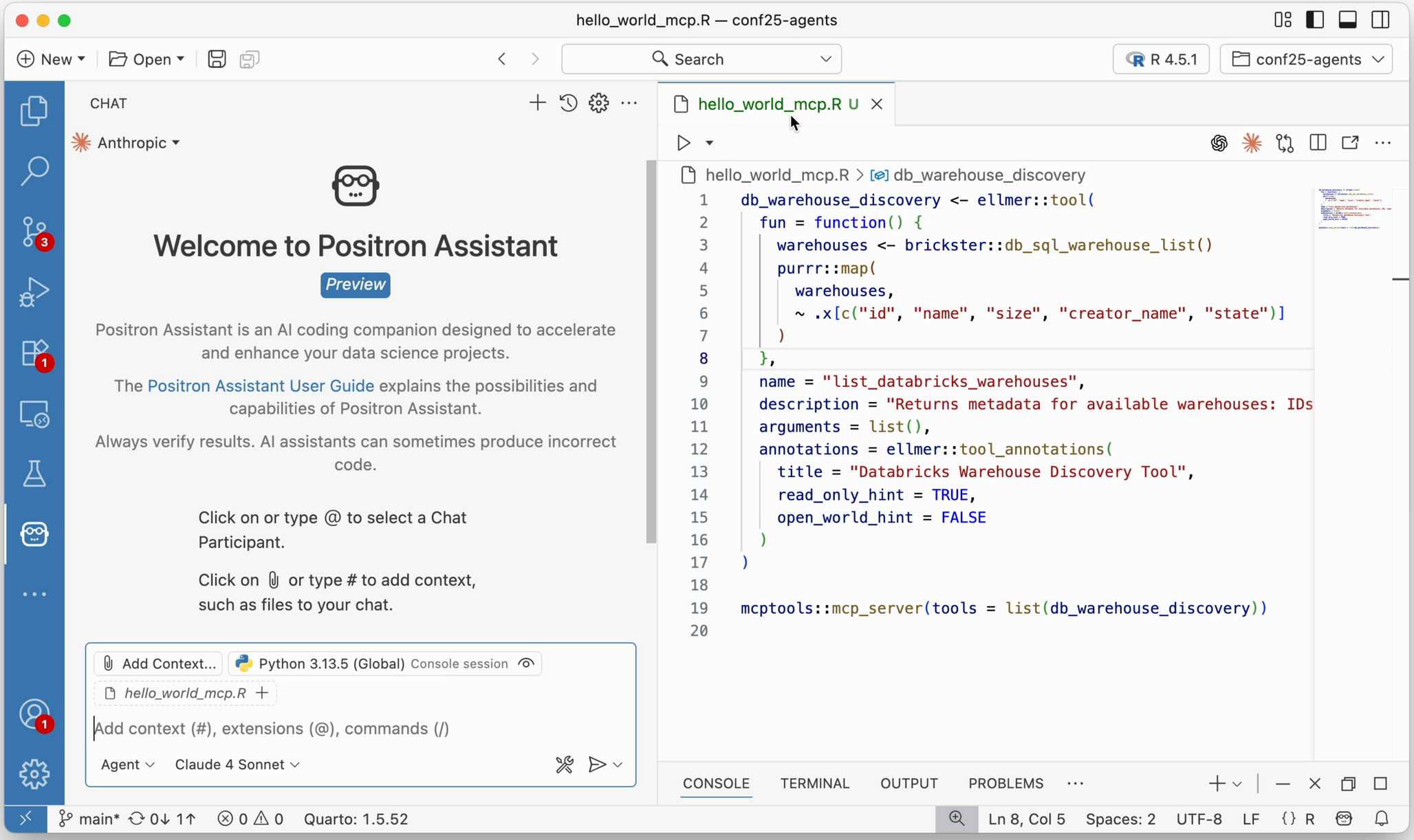
That's it... really...

```
db_warehouse_discovery ← ellmer::tool(
  fun = function() {
    warehouses ← brickster::db_sql_warehouse_list()
    purrr::map(
      warehouses,
      ~ .x[c("id", "name", "size", "creator_name", "state")]
    )
  },
  name = "list_databricks_warehouses",
  description = "Returns metadata for available warehouses: IDs, names, state, siz
  arguments = list(),
  annotations = ellmer::tool_annotations(
    title = "Databricks Warehouse Discovery Tool",
    read_only_hint = TRUE,
    open_world_hint = FALSE
  )
)
```

```
mcptools::mcp_server(tools = list(db_warehouse_discovery))
```

MCP server with my tools please

Add MCP to Positron Assistant



mcp.json

Where Positron assistant looks for servers

~/Library/Application Support/Positron/User/mcp.json

```
{
    "servers": {
        "brickster-example": {
            "type": "stdio",
            "command": "Rscript",
            "args": [
                "/Users/zachary.davies/Documents/projects/conf25-agents/hello_world_mcp.R"
            ]
        }
    },
    "inputs": []
}
```

mcp.json

Where Positron assistant looks for servers

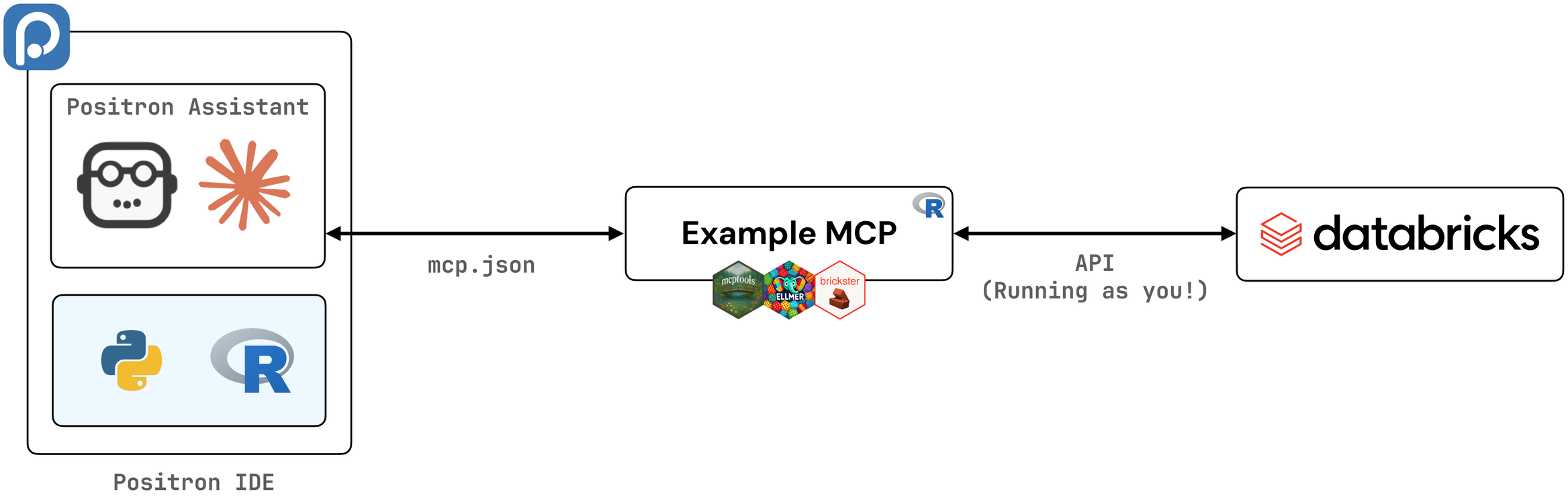
~/Library/Application Support/Positron/User/mcp.json

```
{
    "servers": {
        "brickster-example": {
            "type": "stdio",
            "command": "Rscript",
            "args": [
                "/Users/zachary.davies/Documents/projects/conf25-agents/hello_world_mcp.R"
            ]
        }
    },
    "inputs": []
}
```

“Run this script to start the MCP server”

Add MCP to Positron Assistant

What just happened?



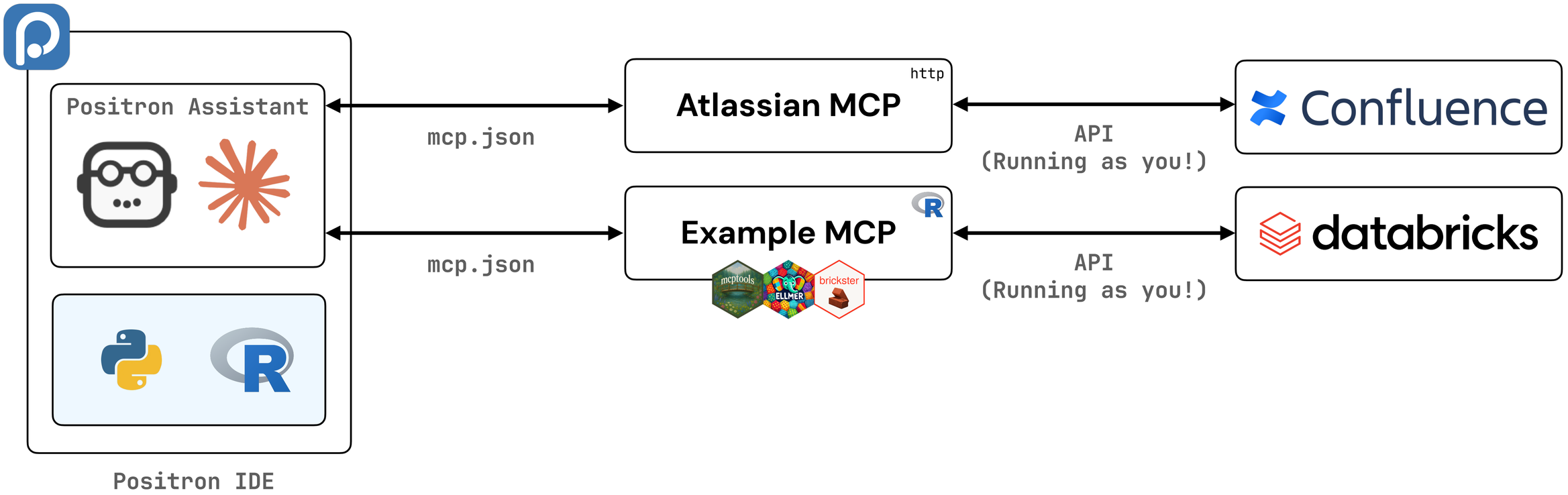
Adding Confluence

Via Atlassian MCP server

~/Library/Application Support/Positron/User/mcp.json

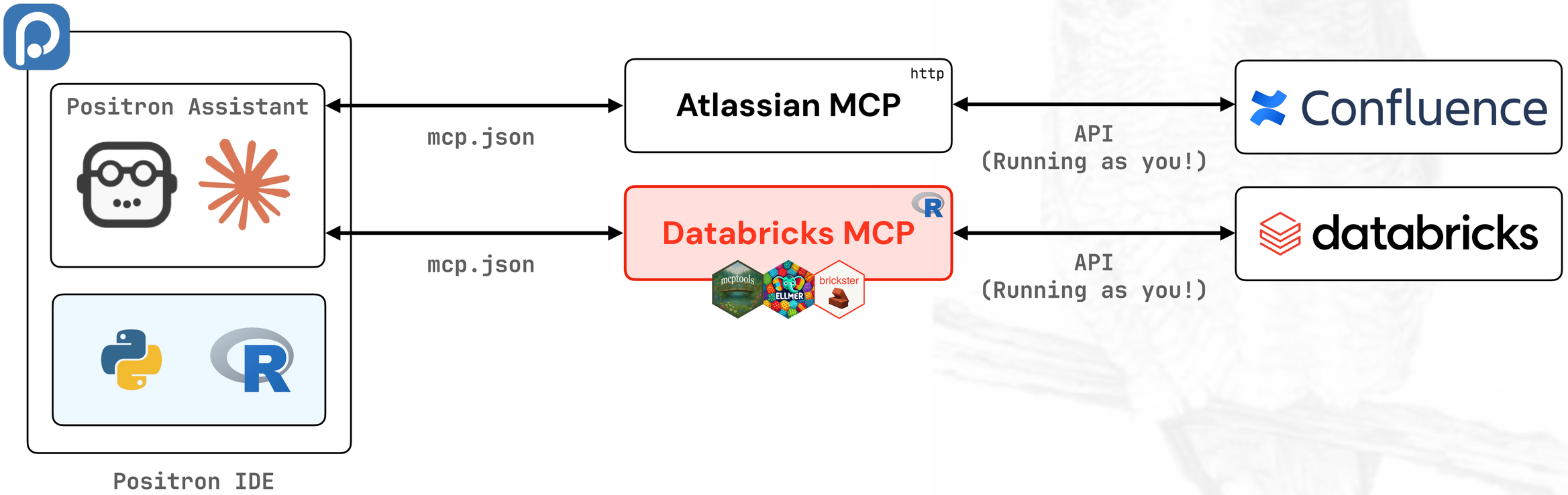
```
{
    "servers": {
        "brickster-example": {
            "type": "stdio",
            "command": "Rscript",
            "args": [
                "/Users/zachary.davies/Documents/projects/conf25-agents/hello_world_mcp.R"
            ]
        },
        "atlassian": {
            "url": "https://mcp.atlassian.com/v1/sse",
            "type": “http"
        }
    },
    "inputs": []
}
```

Adding Confluence



Extend Example MCP

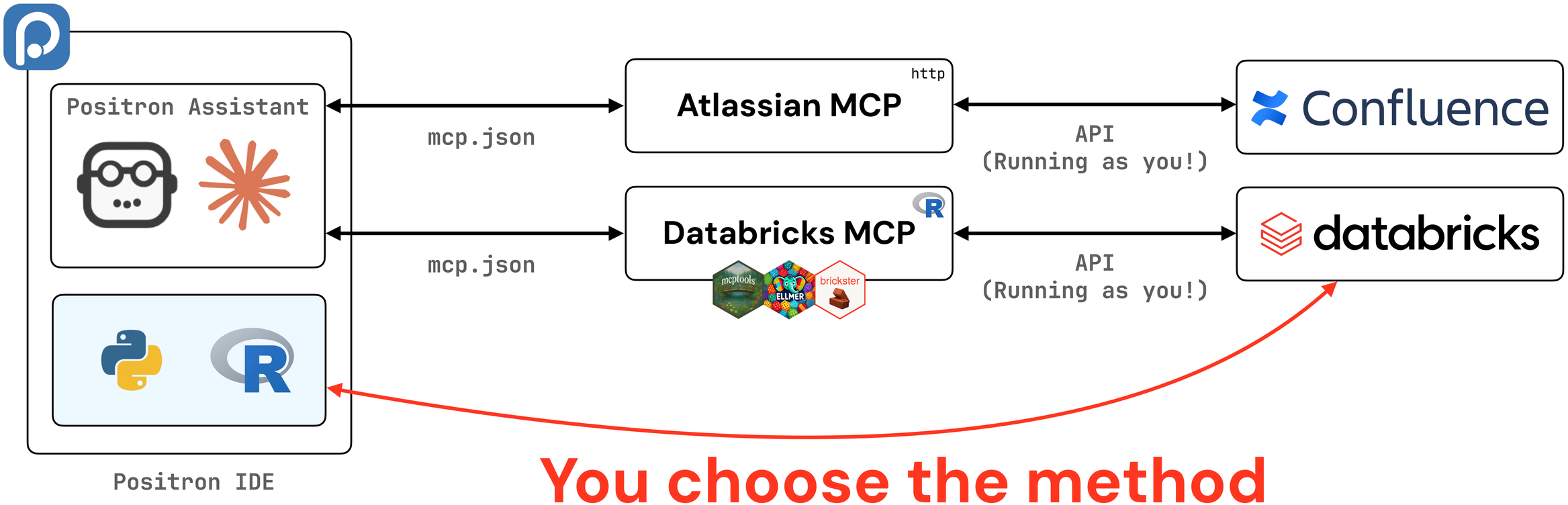
Actual “Rest of the owl” moment



{brickster} MCP code available in talk appendix

The final piece...

Code execution



You choose the method

e.g. brickster, pyspark

So... how realistic was that?

Data Security

- 3rd party providers (e.g. Anthropic)
- Assistant must respect data governance policies

Resource Constraints

- Tokens are expensive
- Do you have enough compute?

Complexity

- Multiple systems involved
- Authentication

So... how realistic was that?

Data Security

- 3rd party providers (e.g. Anthropic)
- Assistant must respect data governance policies

Resource Constraints

- Tokens are expensive
- Do you have enough compute?

Complexity

- Multiple systems involved
- Authentication

databricks

- Unity Catalog
- Serverless Compute
- AI Gateway

Try this yourself

What you'll need

Languages

Databricks

Free Edition Account

Positron Assistant

Positron IDE
Anthropic Token

Atlassian

Confluence

Slides & Code

Everything is optional

Research MCP servers that are relevant for how **you** work


databricks